

Linux File permissions & Access Control Audit

Project description

As a security professional supporting an internal research team, I conducted an authorization and access-control audit on the `/home/researcher2/projects` directory. The existing file system permissions did not align with organizational authorization policies and exposed critical assets to unauthorized modifications. Using Linux command-line tools, I audited the directory structure, deconstructed permission strings, and applied the principle of least privilege using `chmod` to restrict access across users, groups, and others.

Check file and directory details

To perform an initial security audit of the `projects` directory, I used the `ls` command combined with flags to inspect permission flags, ownership, and hidden files:

```
researcher2@5d738f0f927b:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 ..
-rw--w---- 1 researcher2 research_team  46 Dec  2 15:27 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  2 15:27 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Dec  2 15:27 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  2 15:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_t.txt
```

- **Command Explanation:** The `ls -la` command lists all contents in long listing format (`-l`) and includes hidden files (`-a`), which start with a leading period (`.`) such as `.project_x.txt`.
- **Output Analysis:** The audit revealed one subdirectory (`drafts`), one hidden file (`.project_x.txt`), and five project text files. The 10-character string in the first column defines the current access control configuration for each entity.

Describe the permissions string

Taking `project_t.txt` (`-rw-rw-r--`) as an example, the 10-character permission string is deconstructed into four distinct components:

- **1st Character (File Type):** A hyphen (`-`) indicates a regular data file, whereas a `d` indicates a directory.

- **2nd–4th Characters (User / Owner):** `rw-` indicates the owner (`researcher2`) has read (`r`) and write (`w`) permissions, but no execute (`x`) permissions.
- **5th–7th Characters (Group):** `rw-` indicates members of the `research_team` group have read (`r`) and write (`w`) permissions.
- **8th–10th Characters (Other / World):** `r--` indicates all other system users only possess read (`r`) access and are restricted from writing or executing the file.

Change file permissions

Organizational policy dictates that the "other" category must never have write access to internal project files. The audit revealed `project_k.txt` had permissions of `-rw-rw-rw-`, violating this policy.

```
researcher2@5d738f0f927b:~/projects$ chmod o-w project_k.txt
researcher2@5d738f0f927b:~/projects$ ls -la project_k.txt
-rw-rw-r-- 1 researcher2 research_team 46 Dec  2 15:27 project_k.txt
```

- **Command Explanation:** The `chmod o-w project_k.txt` command removes (`-`) the write permission (`w`) specifically from other users (`o`).
- **Verification:** Running `ls -la` confirms that `project_k.txt` was successfully updated to `-rw-rw-r--`, securing it against unauthorized public modifications.

Change file permissions on a hidden file

The file `.project_x.txt` is an archived asset that requires heightened protection. The organization required that no users have write permissions, while `researcher2` (user) and `research_team` (group) retain read-only permissions.

```
researcher2@3213bbc1d047:~/projects$ chmod u-w,g-w,g+r .project_x.txt
researcher2@3213bbc1d047:~/projects$ ls -la .project_x.txt
-r--r----- 1 researcher2 research_team 46 Dec 20 15:36 .project_x.txt
```

- **Command Explanation:** Multiple permission adjustments were chained together with commas: `u-w` strips write permissions from the user, `g-w` strips write permissions from the group, and `g+r` explicitly assigns read permissions to the group.
- **Verification:** The permission string updated from `-rw--w----` to `-r--r-----`, locking the archived hidden file into a secure read-only state for authorized entities.

Change directory permissions

For directories, execute permissions (x) govern the ability to enter and access the directory's contents. Organizational security policy required that access to the `drafts` subdirectory be restricted exclusively to the `researcher2` user.

```
researcher2@5d738f0f927b:~/projects$ chmod g-x drafts
researcher2@5d738f0f927b:~/projects$ ls -la
drwx----- 2 researcher2 research_team 4096 Dec  2 15:27 drafts
```

- **Command Explanation:** `chmod g-x drafts` revokes execute (x) permissions from the `research_team` group.
- **Verification:** The directory permissions changed from `drwx--x---` to `drwx-----`. This guarantees that only the owner (`researcher2`) can navigate into `drafts` and access its underlying contents.

Summary

Through this administrative audit, I inspected, identified, and remediated access-control vulnerabilities across files, hidden files, and directories within the Linux file system. By auditing initial state permissions with `ls -la` and applying surgical permission modifications using `chmod`, I successfully enforced the principle of least privilege. The resulting configuration ensures data integrity and confidentiality across all research assets while strictly limiting system authorization to verified roles.